

M5_Assignment

Danni Martin

2024-11-25

Q2 Avocados

Q2.1 Loading Packages and Data

Write code below that loads fpp3, tidyverse packages. Then load the avocado2020.csv data set using the read_csv() function and save it in an object called avocado.

```
library(fpp3)
```

```
Warning: package 'fpp3' was built under R version 4.4.2
```

```
Registered S3 method overwritten by 'tsibble':
```

```
  method          from  
as_tibble.grouped_df dplyr
```

```
-- Attaching packages ----- fpp3 1.0.1 --
```

```
v tibble      3.2.1    v tsibble      1.1.5  
v dplyr       1.1.4    v tsibbledata 0.4.1  
v tidyr       1.3.1    v feasts      0.4.1  
v lubridate   1.9.3    v fable       0.4.1  
v ggplot2     3.5.1
```

```
Warning: package 'tsibble' was built under R version 4.4.2
```

```
Warning: package 'tsibbledata' was built under R version 4.4.2
```

```
Warning: package 'feasts' was built under R version 4.4.2
```

Warning: package 'fabletools' was built under R version 4.4.2

Warning: package 'fable' was built under R version 4.4.2

```
-- Conflicts ----- fpp3_conflicts --
x lubridate::date()      masks base::date()
x dplyr::filter()        masks stats::filter()
x tsibble::intersect()   masks base::intersect()
x tsibble::interval()    masks lubridate::interval()
x dplyr::lag()            masks stats::lag()
x tsibble::setdiff()      masks base::setdiff()
x tsibble::union()        masks base::union()
```

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats 1.0.0      v readr  2.1.5
v purrr  1.0.2       v stringr 1.5.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter()      masks stats::filter()
x tsibble::interval()  masks lubridate::interval()
x dplyr::lag()          masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
avocado = read.csv("https://jagelves.github.io/Data/avocado2020.csv")
```

Q2.2 Piping1

Use piping to accomplish the following tasks. Start with the avocado object. - remove the observations with a date of 1/1/18 using the filter() function. - use the mutate() and replace() functions to replace observations with a date of 1/7/19 to 1/6/19. - use the mutate() and replace() functions to replace observations with a date of 1/6/20 to 1/5/20. - filter the data so that the geography included is “Richmond/Norfolk” and the type of avocado is “organic”. - select only the date and average_price variables Save the new data into an object called RVA. RVA should have 305 observations and two variables. Include your code below:

```
avocado %>%
  filter(date!="1/1/18")) %>%
  mutate(date = replace(date, date == ("1/7/19"), ("1/6/19"))) %>%
  mutate(date = replace(date, date == ("1/6/20"), ("1/5/20"))) %>%
  filter(geography == "Richmond/Norfolk", type == "organic") %>%
  select(date, average_price) -> RVA
```

Q2.3 Piping2

Use piping to accomplish the following tasks. Start with the RVA object.

- use the mutate() function to coerce the date variable to a date that has the year and week by nesting the mdy() function inside of the yearweek() function.
- coerce the data to a tsibble using the as_tsibble() function.
- filter the period “2019-01-07” to “2020-11-29” by using the filter_index() function. Save the result in an object called RVA_ts. RVA_ts should have 99 observations and two variables. Include your code below:

```
RVA %>%
  mutate(date = yearweek(mdy(date))) %>%
  as_tsibble(index=date) %>%
  filter_index("2019-01-07"~"2020-11-29") -> RVA_ts
```

Q2.4 Train and Test Sets

Create a test and training set and save them into objects called train_RVA and test_RVA*, respectively. The training set should include the period 2019-01-06 to 2020-10-25, and the test set should include the period 2020-11-01 to 2020-11-29. Include the code below:

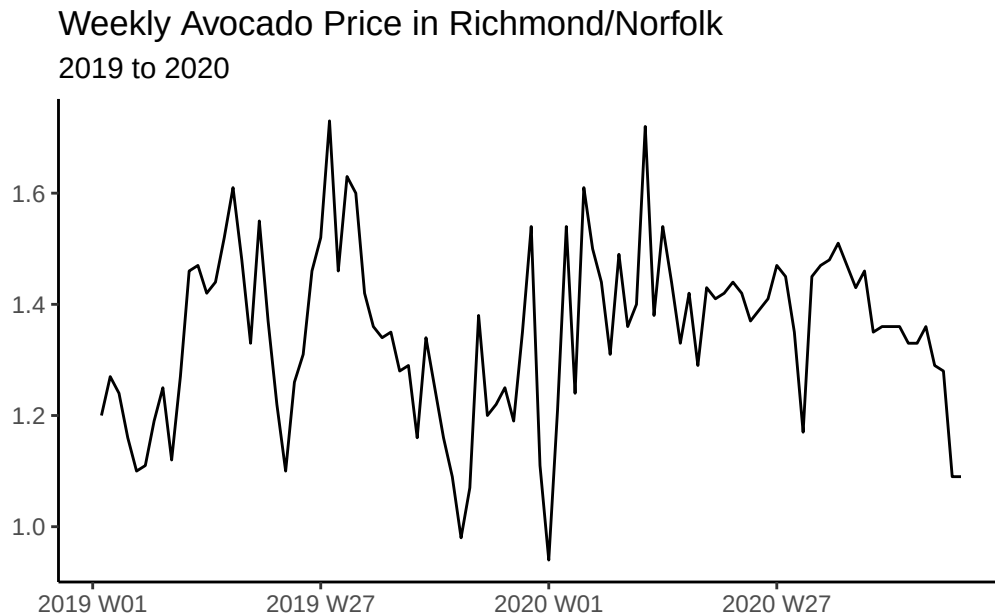
```
RVA_ts %>%
  filter_index("2019-01-06"~"2020-10-25") -> train_RVA

RVA_ts %>%
  filter_index("2020-11-01"~"2020-11-29") -> test_RVA
```

Q2.5 Autoplot

Create a plot of the average price of avocados (using RVA_ts) in Richmond/Norfolk using the autoplot() function. The title should read “Weekly Avocado Price in Richmond/Norfolk”. The subtitle should read “2019 to 2020”. Upload a screenshot of the plot below:

```
RVA_ts %>%
  autoplot(average_price) + theme_classic() +
  labs(x = "",
       y = "",
       title = "Weekly Avocado Price in Richmond/Norfolk",
       subtitle = "2019 to 2020")
```



Q2.6 Fit Models

Fit the following models using the `model()` and `TSLM()` functions. Use the training set.

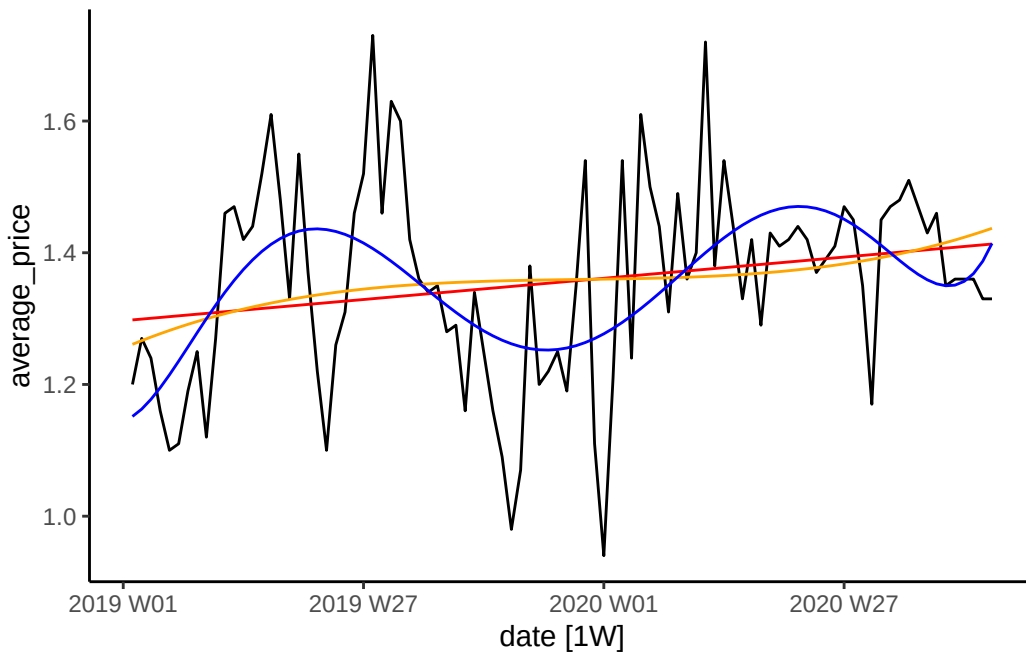
LS: A simple linear model with the `trend()` parameter. LS3: A cubic model with the `trend()`, `trend()^2`, and `trend()^3` parameters. LS6: A hexic model with the `trend()...trend()^6` parameters. Save the result in an object called `fit`. Include your code below:

```
train_RVA %>%
  model(LS=TSLM(average_price~trend()),
        LS3=TSLM(average_price~trend()+I(trend()^2)+I(trend()^3)),
        LS6=TSLM(average_price~trend()+I(trend()^2)+I(trend()^3)
                  +I(trend()^4)+I(trend()^5)+I(trend()^6))) -> fit
```

Q2.7 Plot1

Create a plot that shows the training set and the fitted values of all the models estimated (LS,LS3, and LS6). Use a red line for the LS model, an orange line for the LS3 model, and a blue line for the LS6 model.

```
train_RVA %>% autoplot(average_price) + theme_classic() +  
  geom_line(aes(y = .fitted), col="red",  
            data = augment(fit) %>% filter(`.model`=="LS")) +  
  geom_line(aes(y = .fitted), col="orange",  
            data = augment(fit) %>% filter(`.model`=="LS3")) +  
  geom_line(aes(y = .fitted), col="blue",  
            data = augment(fit) %>% filter(`.model`=="LS6"))
```



Q2.8 gt Package

Load the gt package. Use piping to accomplish the following tasks. Start with the fit object.

- use the accuracy() function to find the model's fit for the training set.
- select the variables .model and RMSE.
- arrange the data using the arrange() function by the RMSE.
- use the gt package to highlight the row (model) with the lowest RMSE.

.model	RMSE
LS6	0.1310198
LS3	0.1486626
LS	0.1492001

```
library(gt)
```

Warning: package 'gt' was built under R version 4.4.2

```
fit %>%
  accuracy() %>%
  select(.model, RMSE) %>%
  arrange(RMSE) %>%
  gt() %>%
  tab_style_body(
    style = cell_fill(color="lightgreen"),
    values = "LS6",
    targets = "row")
```

Q2.9 Information Criterion

Use the fit object to retrieve the AIC, AICc, and BIC. According to the BIC, which model should be preferred? Why?

```
# The LS6 Model as its value is lowest.
fit %>% glance() %>%
  select(".model", "AIC", "AICc", "BIC")
```

```
# A tibble: 3 x 4
  .model  AIC  AICc  BIC
  <chr>  <dbl> <dbl> <dbl>
1 LS    -352. -351. -344.
2 LS3   -348. -348. -336.
3 LS6   -366. -364. -346.
```

Q2.10 CV

Use piping to perform cross-validation. Start with the `train_RVA` object.

- use the `stretch_tsibble()` function with the argument `.init` set to 20 and the `.step` argument set to five.
- estimate the LS, LS3 and LS6 models once again using the `model()` and `TSLM()` functions.
- forecast five periods ($h=5$) using the `forecast()` function.
- use the `accuracy()` function with the `RVA_ts` as only input.

```
train_RVA %>%  
  stretch_tsibble(.init = 20, .step = 5) %>%  
  model(LS=TSLM(average_price~trend()),  
        LS3=TSLM(average_price~trend()+I(trend()^2)+I(trend()^3)),  
        LS6=TSLM(average_price~trend()+I(trend()^2)+I(trend()^3)  
                  +I(trend()^4)+I(trend()^5)+I(trend()^6))) %>%  
  forecast(h=5) %>%  
  accuracy(RVA_ts)
```

```
# A tibble: 3 x 10  
  .model .type      ME  RMSE   MAE   MPE  MAPE  MASE  RMSSE  ACF1  
  <chr>  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  
1 LS     Test    -0.0486 0.191 0.148 -5.01  11.5  0.890  0.938  0.614  
2 LS3    Test    -0.0265 0.319 0.254 -2.77  18.3  1.52   1.57   0.694  
3 LS6    Test    -0.0998 1.26  0.756 -7.03  56.6  4.53   6.22   0.610
```

```
[1] 4
```

The `echo: false` option disables the printing of code (only output is displayed).